BIRZEIT UNIVERSITY

Faculty of Engineering & Technology

Electrical & Computer Engineering Department

Digital Systems – ENCS2340

Verilog HDL Project

Design and Implementation of a Simple ALU using Verilog HDL

Instructor : DR.Ali Abdo

Student Name: Anis DarHammouda

Student No: 1230834

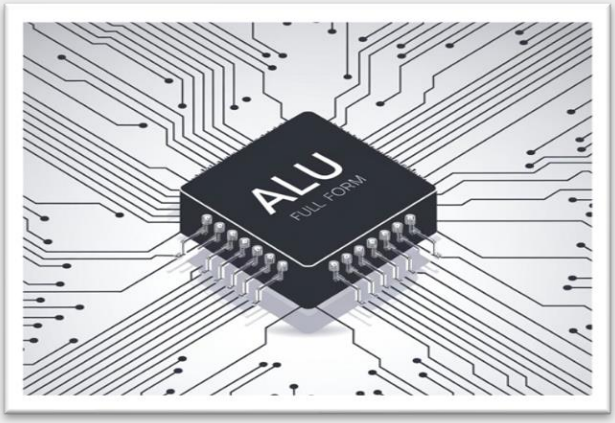
Section: 3

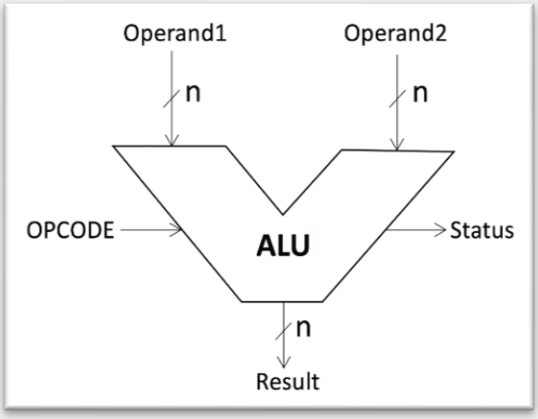
Table of Contents

Abstract: 3
Bitwise OR (4 Bit): 4
Code implementation: ---------- 4
Symbol : ---------- 4
Simulation result: ---------- 5
Bitwise AND (4 Bit): 6
Code implementation: 6
Symbol ---------- 6
Simulation result: ---------- 7
Half Adder: 8
Code implementation: ---------- 8
Symbol: ---------- 8
Simulation result: ---------- 9
Full Adder: 10
Code implementation: ---------- 10
Symbol ---------- 11
Simulation result: ---------- 12
4 – Bit Adder: 13
Code implementation: ---------- 13
Symbol: ---------- 14
Simulation result: ---------- 15
4 – Bit Subtractor: 16
code implementation: ---------- 17
Symbol: ---------- 18
Simulation result: ---------- 19
ALU using DataFlow: 20
code implementation: ---------- 20
Symbol: ---------- 21
Simulation result: ---------- 22
block diagram of ALU_dataFlow: ---------- 23
ALU using Behavioral: 24
Code implementation: ---------- 24
Symbol ---------- 25
Simulation result: ---------- 26
Block diagram of ALU_Behavioral: ---------- 27
Conclusions: 28

Abstract:

ALU(Arithmetic logic unit) is a combinational digital circuit that performs arithmetic and bitwise operations on integer binary numbers. This is in contrast to a floating-point unit (FPU), which operates on floating point numbers. It is a fundamental building block of many types of computing circuits, including the central processing unit (CPU) of computers, FPUs, and graphics processing units (GPUs).





In this project, we designed a simple ALU with four basic arithmetic and logic operations: addition, subtraction, bitwise AND, and bitwise OR using Verilog Hardware Description Language (HDL).

Bitwise OR (4 Bit):

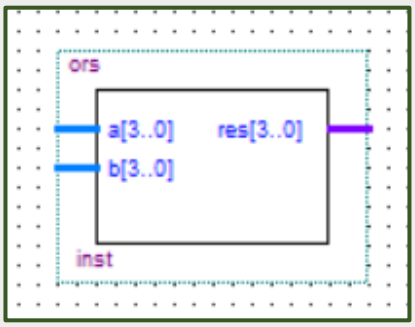
Code implementation:

```
module ors(a,b,res);
input [3:0] a,b;     // inputs a & b each of them is 4 bits
output [3:0] res;    // the result of ORing inputs 4 bit too
or (res[0],a[0],b[0]);
or (res[1],a[1],b[1]);
or (res[2],a[2],b[2]);
or (res[3],a[3],b[3]);
endmodule
```

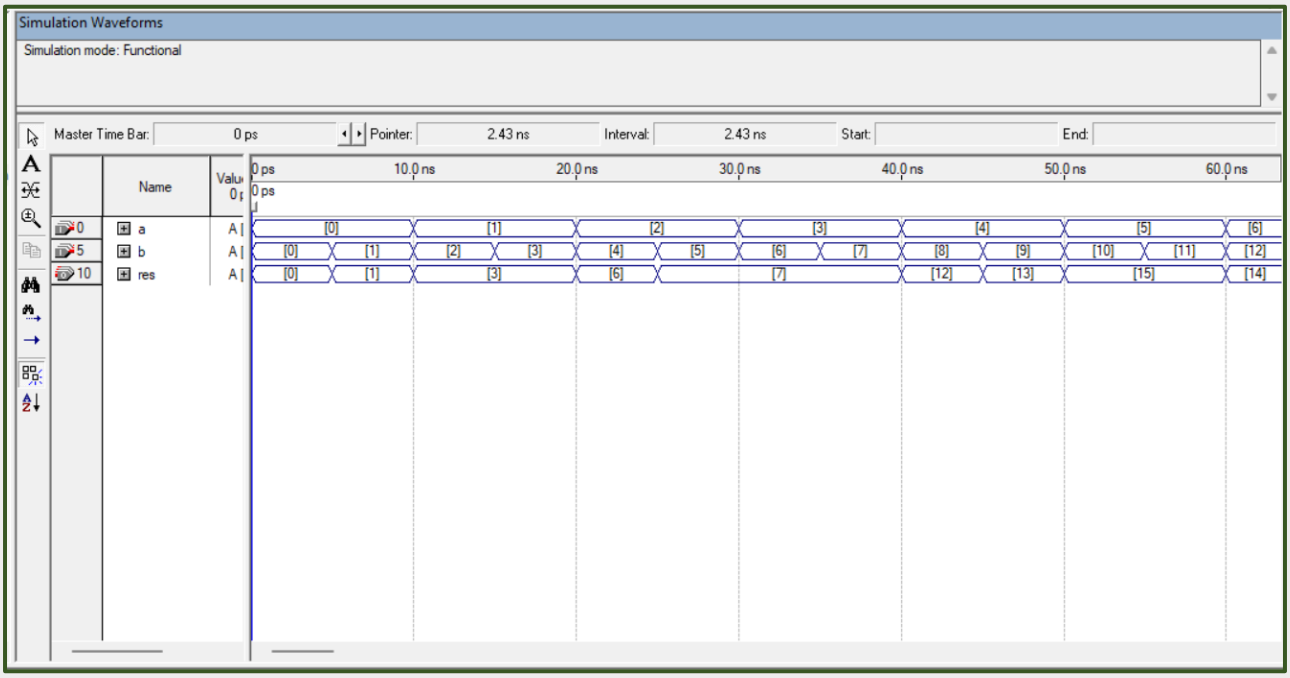
I have named the module of Bitwise OR as “ors”. Using [3:0] inputs a(first 4 bit input), b(second 4 bit input) and the [3:0] output (4 bit) “res” which is the result of ORing the two inputs.

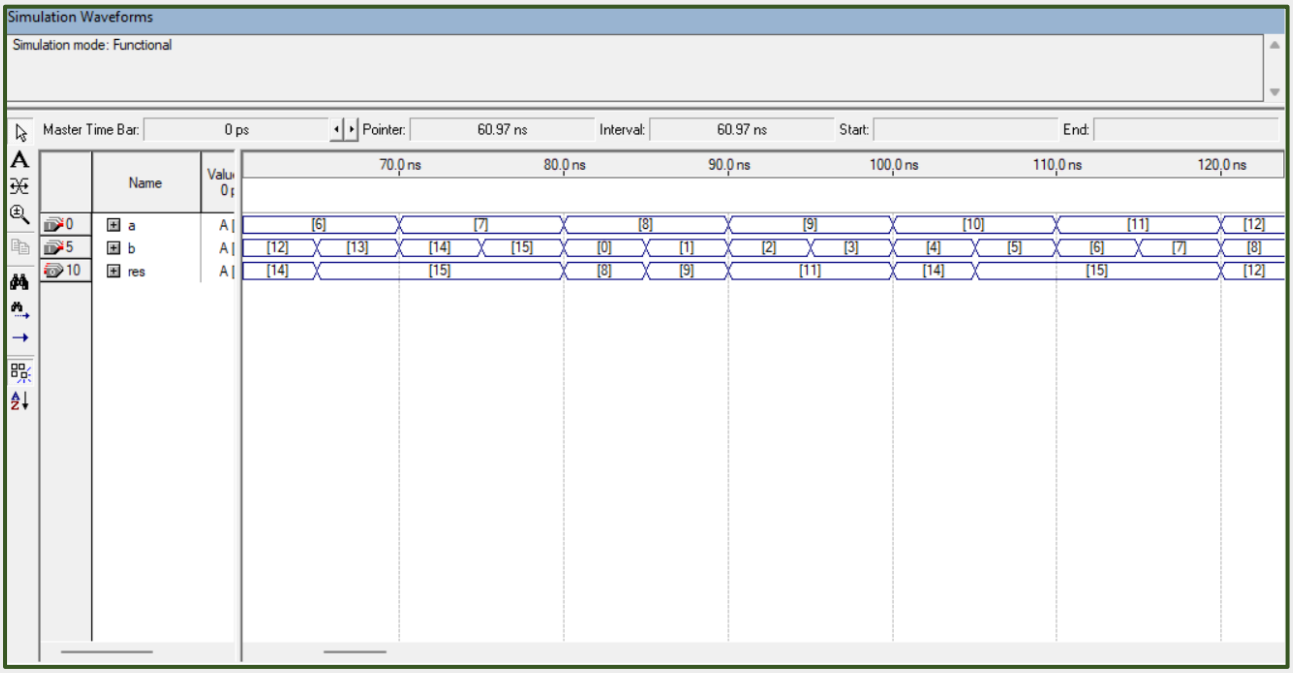
Using structural modeling(Gate – level modeling) as shown in the picture above, by using 4 OR gates to ORing each bit.

Symbol :



Simulation result:





Test case:

For example take a = 9 = $(1001)_2$ and b = 3 = $(0011)_2$:

ORing 9 & 3: a_0 = 1 OR b_0 =1 , res_0 = 1

a_1 = 0 OR b_1 =1 , res_1 = 1

a_2 = 0 OR b_2 =0 , res_2 = 0

a_3 = 1 OR b_3 =0 , res_3 = 1 so res(result) = $(1011)_2$ = 11 which the same in the second picture above.

Bitwise AND (4 Bit):

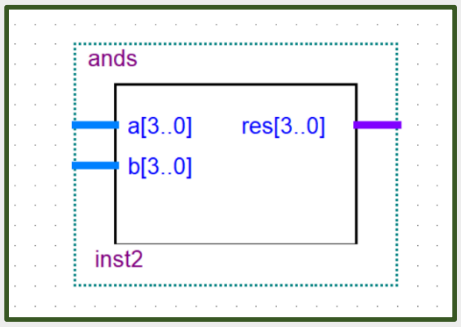
Code implementation:

```
module ands(a,b,res);
input [3:0] a,b; // inputs a & b each of them is 4 bits
output [3:0] res;  // the result of ANDing inputs 4 bit too
and (res[0],a[0],b[0]);
and (res[1],a[1],b[1]);
and (res[2],a[2],b[2]);
and (res[3],a[3],b[3]);
endmodule
```

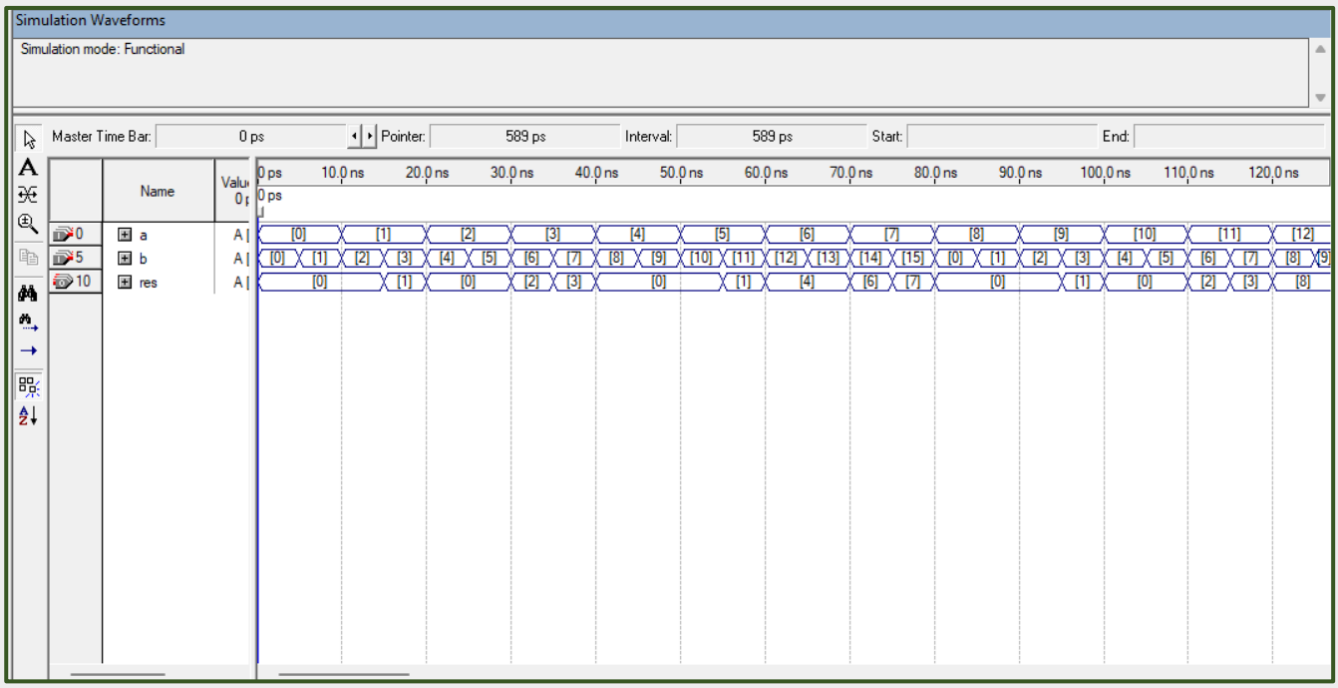
I have named the module of Bitwise or as “ands”, using [3:0] inputs a(first 4 bit input), b(second 4 bit input) and the [3:0] output (4 bit)“res” which is the result of ANDing the two inputs.

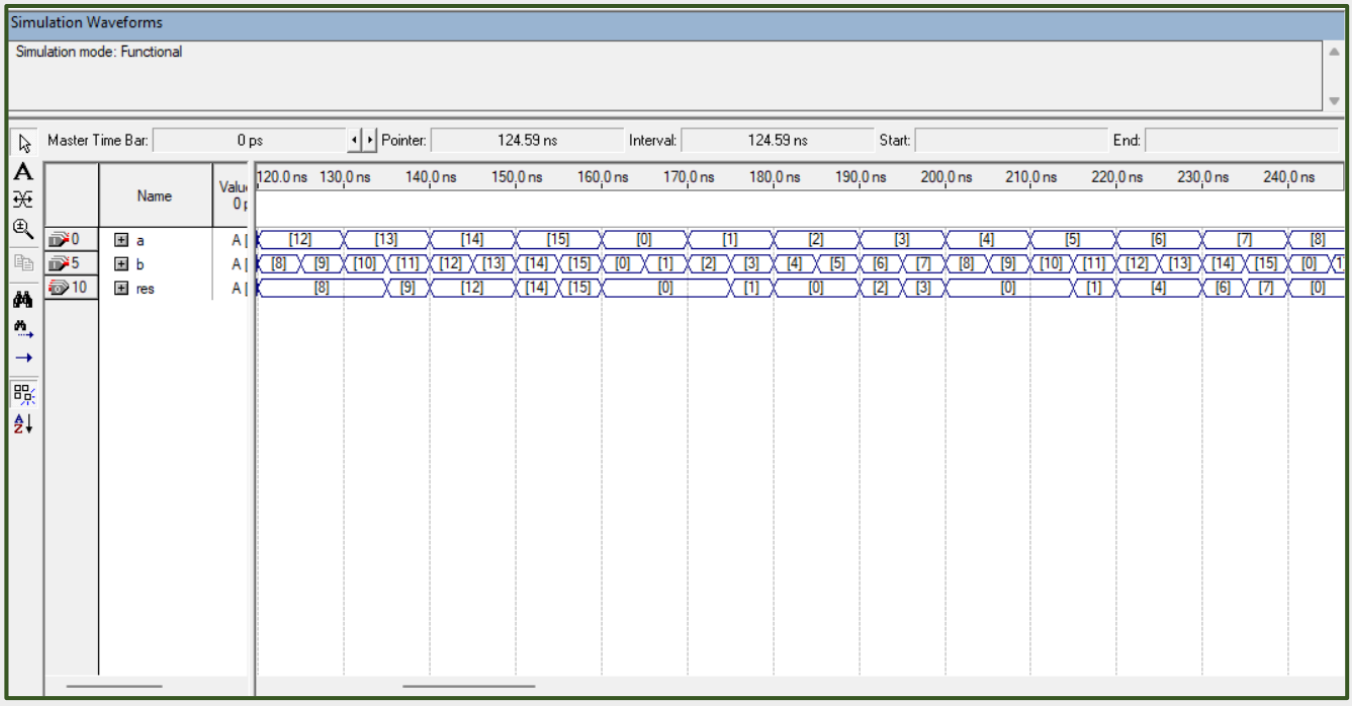
Using structural modeling(Gate – level modeling) as shown in the picture above, by using 4 AND gates to ANDing each bit.

Symbol:



Simulation result:





Test case:

For example take a = 13 = $(1101)_2$ and b = 11 = $(1011)_2$:

ANDing 13 & 11: a_0 = 1 AND b_0 =1 , res_0 = 1

a_1 = 0 AND b_1 = 1 , res_1 = 0

a_2 = 1 AND b_2 = 0 , res_2 = 0

a_3 = 1 AND b_3 = 1 , res_3 = 1 so res(result) = $(1001)_2$ = 9 which the same in the second picture above.

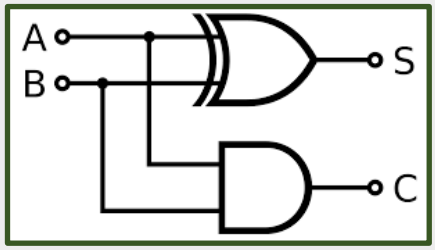
Half Adder:

Code implementation:

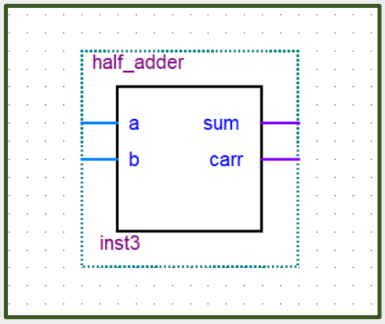
```
module half_adder(a,b,sum,carr);
input a,b;        // the inputs and each of them is one bit
output sum,carr;    // the sum of two inputs and the carry
xor(sum,a,b);
and(carr,a,b);

endmodule
```

I have implement the half adder using structural modeling(Gate-level Modeling) with XOR gate it get the sum of the two inputs a & b, and an AND gate to get the carry out of the procedure of adding two bits.

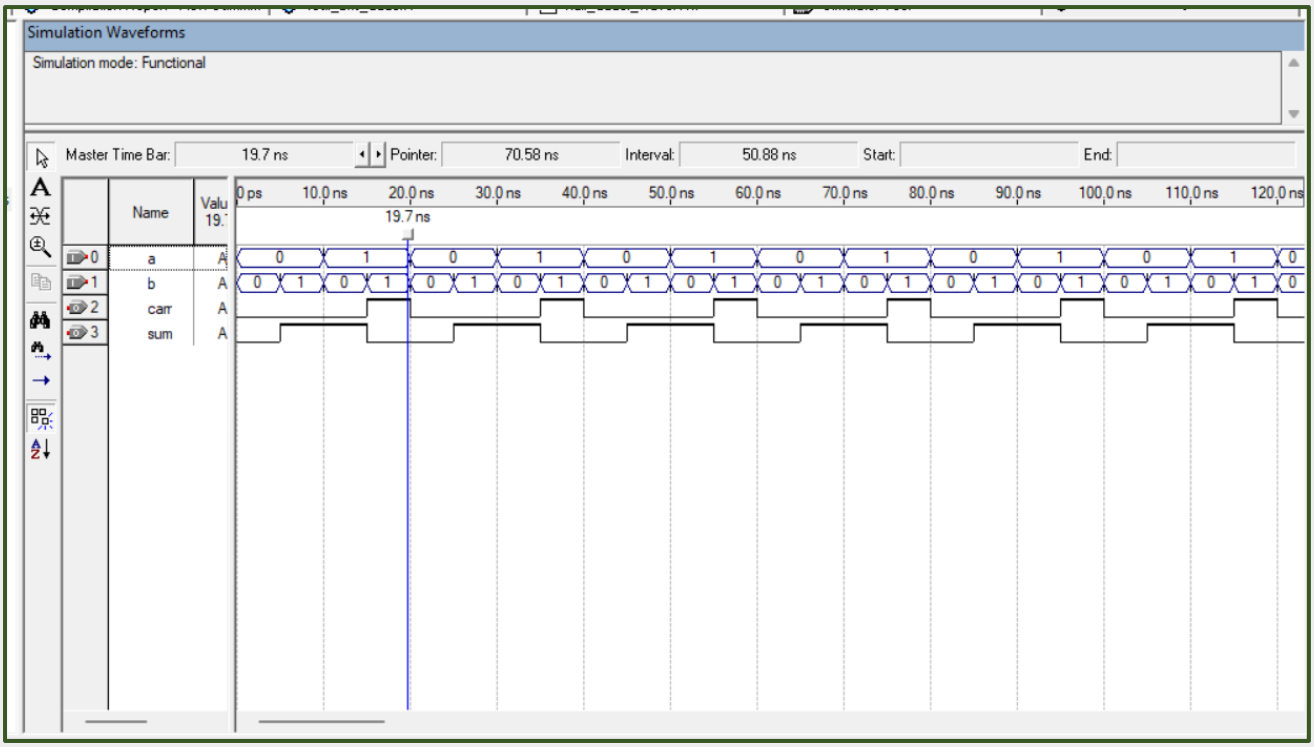
(I have made the half adder to implement the full adder. However full adder can implemented without using half adder).



Symbol:



Simulation result:



Test case:

For example take a = 1 and b = 1:

In binary numbers we know that 1 + 1 = 10 (carry = 1 , sum = 0)

1 XOR 1 = 0 → sum = 0

1 AND 1 = 1 → carry = 1

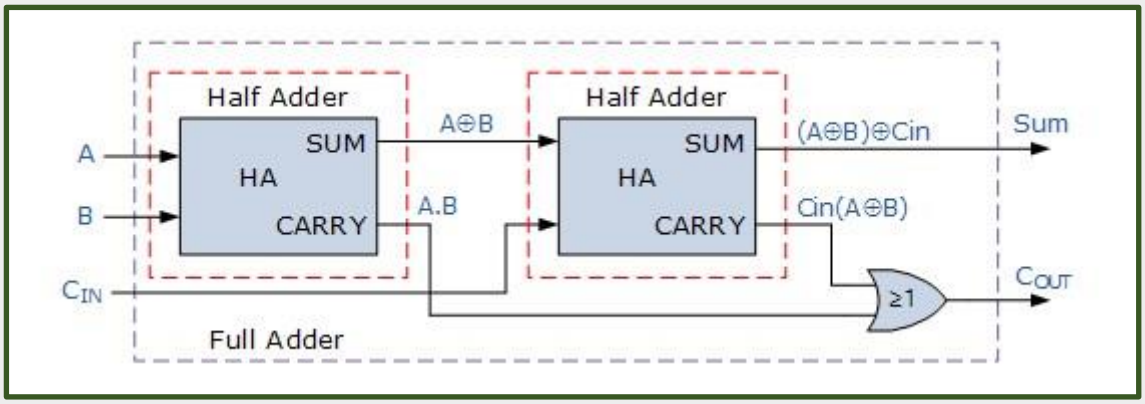
So the result is same as shown in the picture above.

Full Adder:

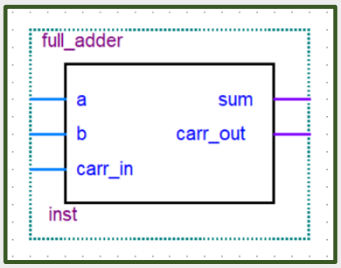
```
module full_adder(a,b,carr_in,sum,carr_out);
input a,b,carr_in;      // the inputs a, b and carry in "carr_in"
output sum,carr_out;    // the sum of inputs and the carry out
wire w1,w2,w3;
// calling half adders
half_adder f1(a,b,w1,w2);
half_adder f2(w1,carr_in,sum,w3);
or or1(carr_out,w2,w3);
endmodule
```

Code implementation:

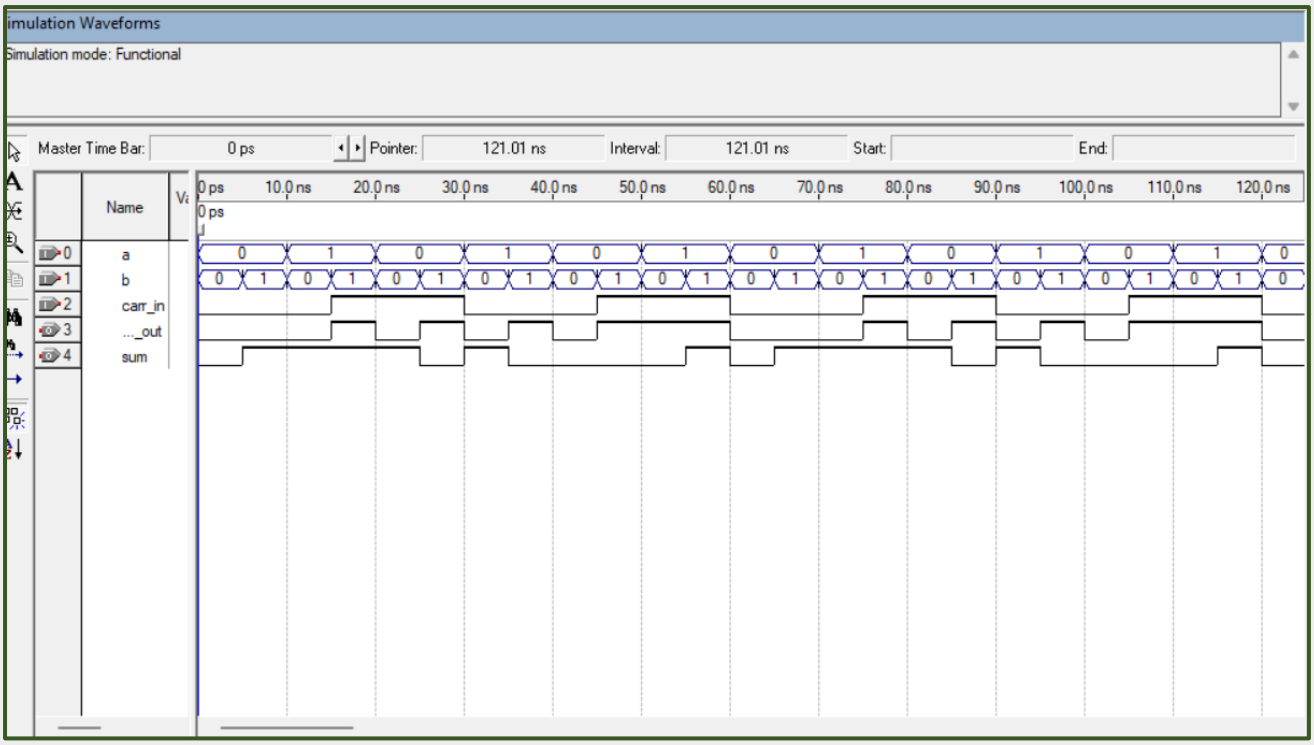
As shown in the above picture, I have used 2 half adders (note: full adder can be implemented without half adders). First half adder takes the inputs a & b and it outputs the sum of a & b in wire w1, and the carry in wire w2. Second half adder takes wire w1(sum of a & b) and carry in as inputs and it outputs are the sum(final sum of the 3 bits) and a carry on wire w3.Finally, OR gate (with w2,w3 as inputs) to get the final carry out of the procedure of adding the 3 bits.


Half Adder
SUM
HA
CARRY
Half Adder
SUM
HA
CARRY
A
B
CIN
A⊕B
A.B
(A⊕B)⊕Cin
Cin(A⊕B)
Sum
COUT
≥1
Full Adder

Symbol:


full_adder
a
b
carr_in
sum
carr_out
inst

Simulation result:



Test case:

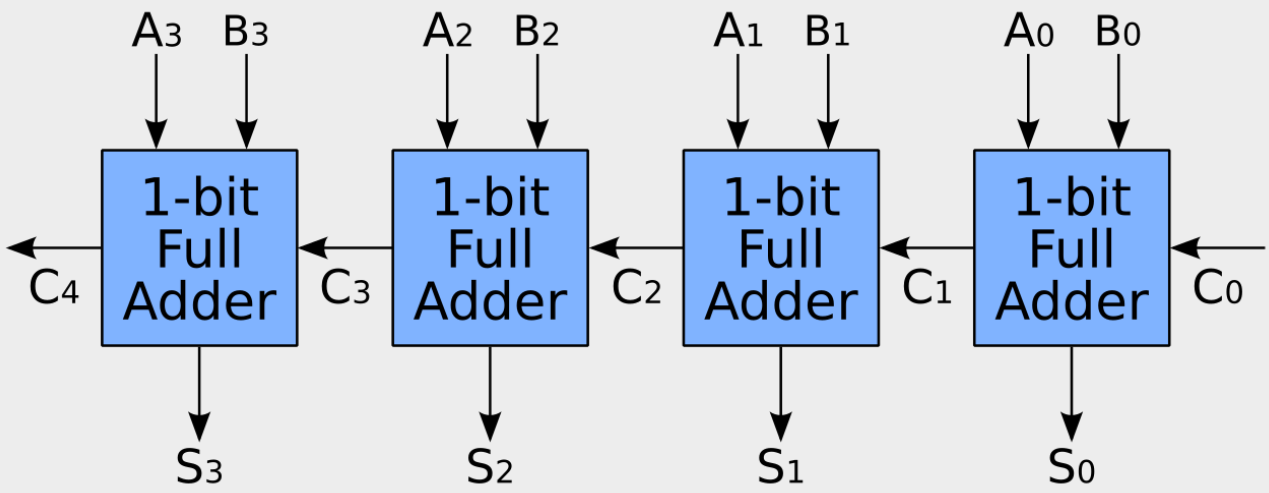
For example take a = 1 ,b = 1 and carry in = 1:
In binary numbers we know that 1 + 1 + 1 = 11 (carry = 1 , sum = 1)
When a & b enter the first half_adder the result will be sum in wire w1 which equal 1 and the carry in wire w2 which equal 1 too.
a XOR b = 1 XOR 1 = 1
Then when wire w1(which has the sum of a & b from first half adder) & carry in enter the second half adder the result is the final sum of the three bits(a, b & carry in) and carry out in wire w3.
W1 XOR carry in = 0 XOR 1 = sum = 1
Finally the carry out from second half adder enter an OR gate with the carry out from first half adder(which in wire w2) and the result of this OR gate is the final carry out of adding the three bits.
1 OR 1 = 1 → carry out = 1
So the result is same as shown in the picture above.

4 – Bit Adder:

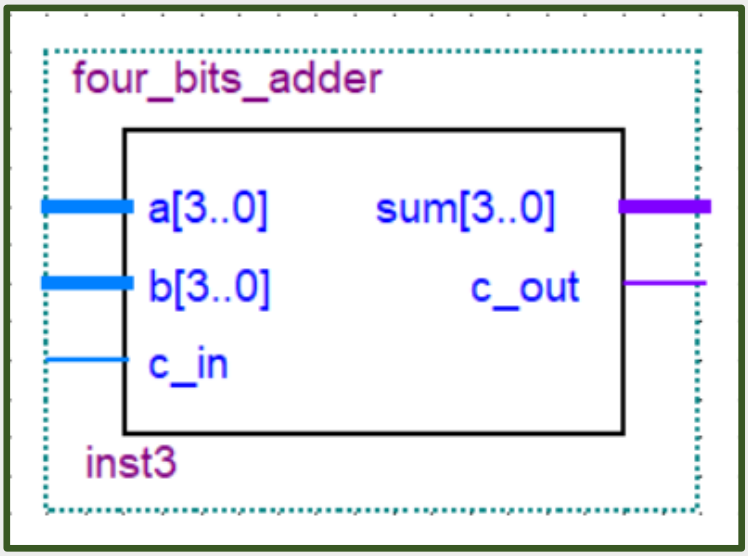
Code implementation:

```
module four_bits_adder(a,b,c_in,sum,c_out);
input [3:0] a,b;                          // the two 4 bit numbers to find their sum
input c_in;                               // carry in in the first full adder usually equal zero
wire c1,c2,c3;                            // wires to represent each carry from each full adder
output [3:0] sum;                         // the result of sum
output c_out;                             // final carry out of the sum

full_adder(a[0],b[0],c_in,sum[0],c1);     // calling full adders which built before
full_adder(a[1],b[1],c1,sum[1],c2);
full_adder(a[2],b[2],c2,sum[2],c3);
full_adder(a[3],b[3],c3,sum[3],c_out);

endmodule
```

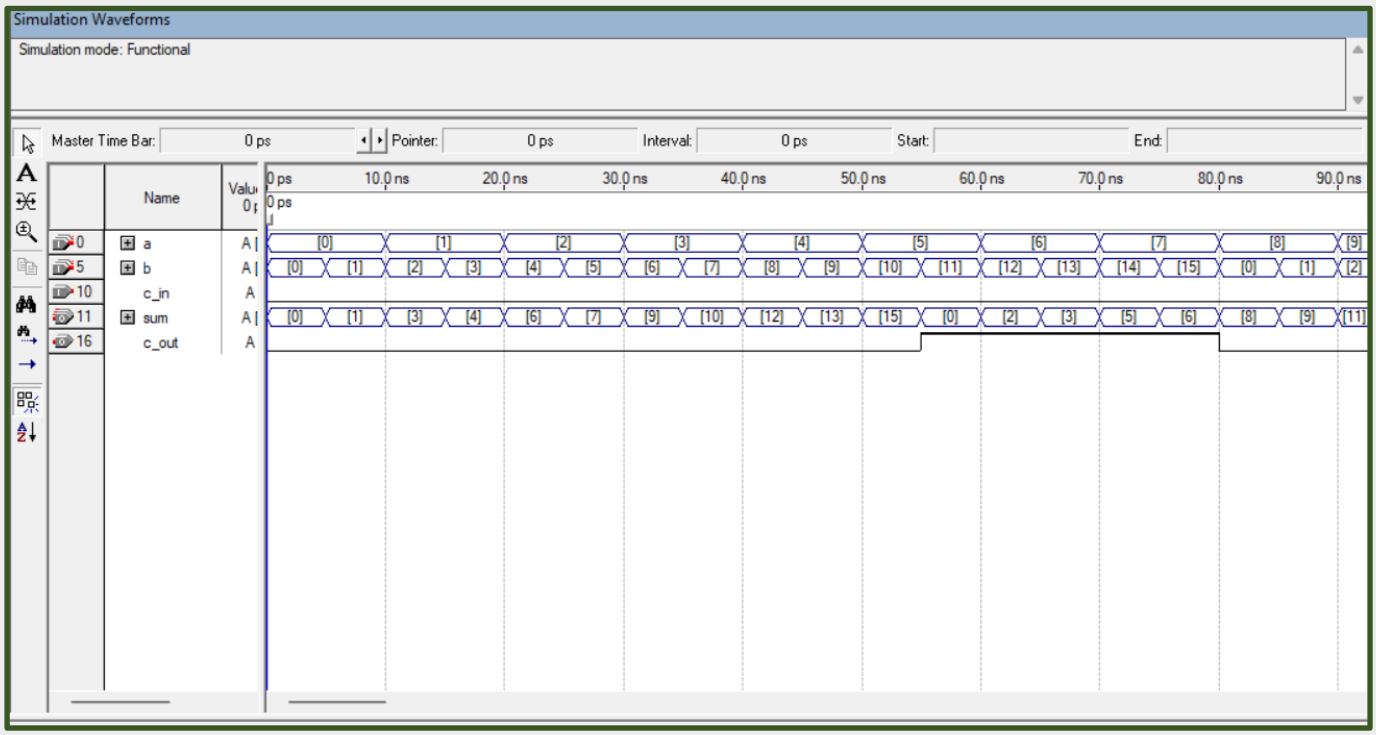
It is Ripple adder, I have used 4 full adders to implement 4 bit adder. It takes a & b as 4 bit inputs and it outputs are sum and carry out of adding the 4 bits. Wires c1,c2,c3 are for carries out form each full adder starting from the firs full adder and take its carry out in c1 , then using c1 as carry_in in the first full adder , and c2 is the carry out from the first full adder and it is the carry_in in the second full adder and so on until we get the carry out from last full adder and it is the carry out of the addition procedure. Output sum is 4 bit each bit is the result from adding Corresponding bits from inputs a & b(each sum bit from one full adder).

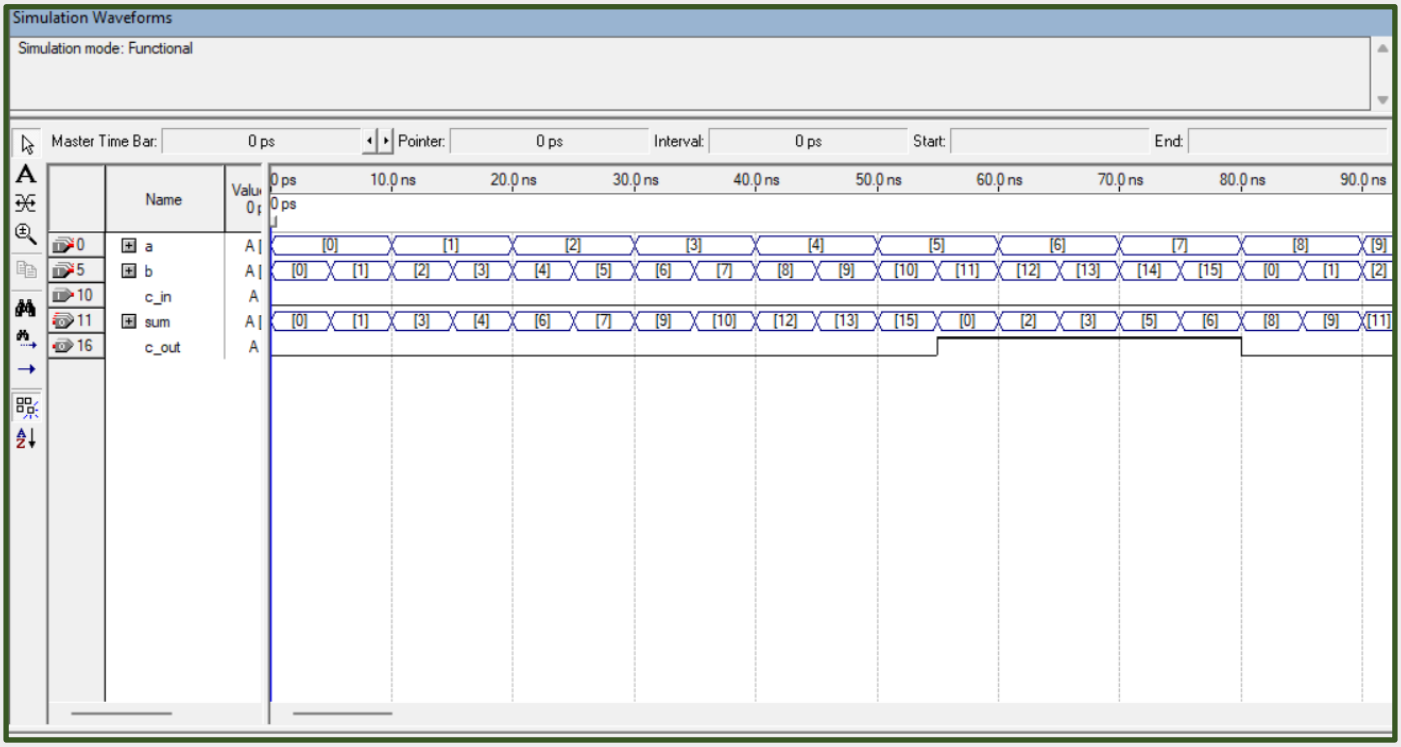

A3 B3
A2 B2
A1 B1
A0 B0
1-bit Full Adder
1-bit Full Adder
1-bit Full Adder
1-bit Full Adder
C4
C3
C2
C1
C0
S3
S2
S1
S0

Symbol:


four_bits_adder
a[3..0]
b[3..0]
c_in
sum[3..0]
c_out
inst3

Simulation result:





Test case:

Let's take a = 4 = $(0100)_2$ & b = 9 = $(1001)_2$:

adding 4 & 9: $a_0 = 0 + b_0 = 1 + C0 = 0$, $Sum_0 = 1$,$C_1 = 0$

$a_1 = 0 + b_1 = 0 + C_1 = 0$, $Sum_1 = 0$,$C_2 = 0$

$a_2 = 1 + b_2 = 0 + C_2 = 0$, $Sum_2 = 1$,$C_3 = 0$

$a_3 = 0 + b_3 = 1 + C_3 = 0$, $Sum_3 = 1$,$C_4 = 0$ so sum = $(1101)_2$ = 13 and carry out = 0 which the same in the first picture above.

Another Test case:

Let's take a = 12 = $(1100)_2$ & b = 8 = $(1000)_2$:

adding 4 & 9: $a_0 = 0 + b_0 = 0 + C0 = 0$, $Sum_0 = 0$,$C_1 = 0$

$a_1 = 0 + b_1 = 0 + C_1 = 0$, $Sum_1 = 0$,$C_2 = 0$

$a_2 = 1 + b_2 = 0 + C_2 = 0$, $Sum_2 = 1$,$C_3 = 0$

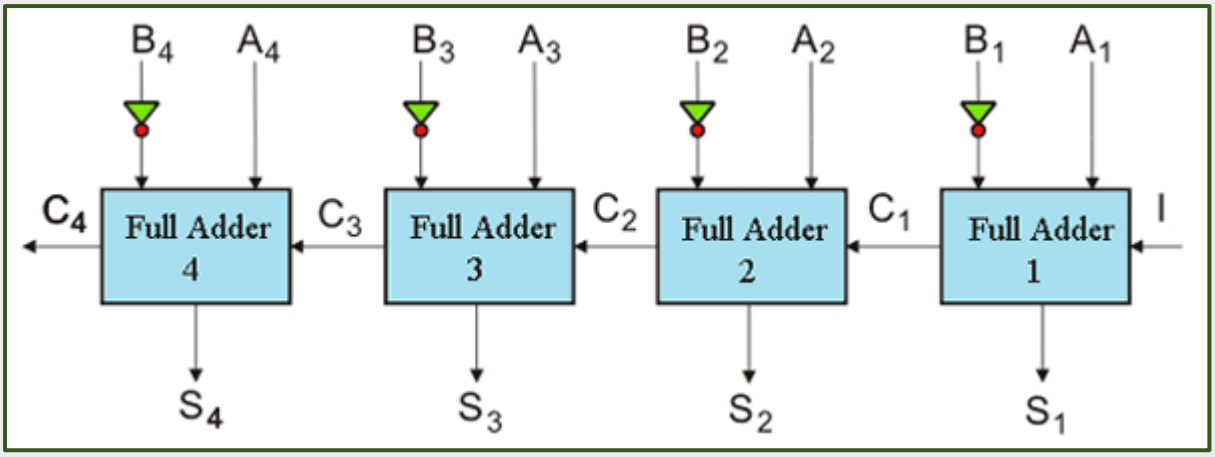
$a_3 = 1 + b_3 = 1 + C_3 = 0$, $Sum_3 = 0$,$C_4 = 1$ so sum = $(0100)_2$ = 4 and carry out = 1 which the same in the second picture above.

4 – Bit Subtractor:

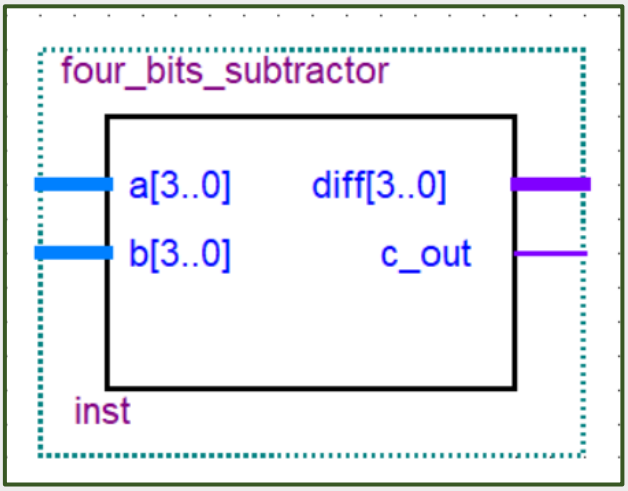
code implementation:

```
module four_bits_subtractor(a,b,diff,c_out);
input [3:0] a,b;
wire c1,c2,c3;                // wires for carries from each full adder
wire [3:0]b_complement;           // to get complement of b
output [3:0] diff;               //difference
output c_out;

not (b_complement[0],b[0]);
not (b_complement[1],b[1]);
not (b_complement[2],b[2]);
not (b_complement[3],b[3]);
full_adder(a[0],b_complement[0],1,diff[0],c1);      // carry_in in the first full adder equal 1
full_adder(a[1],b_complement[1],c1,diff[1],c2);
full_adder(a[2],b_complement[2],c2,diff[2],c3);
full_adder(a[3],b_complement[3],c3,diff[3],c_out);

endmodule
```

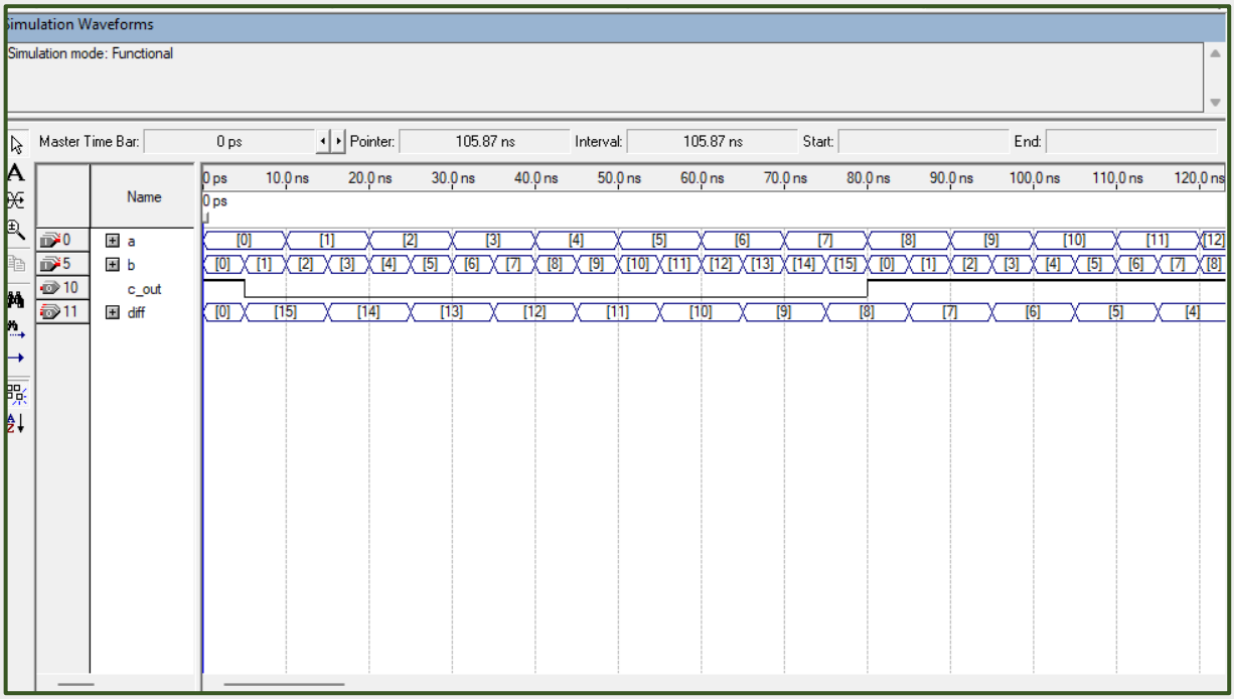
As we know in subtracting binary numbers first step is to get the 2's complement of the subtrahend (second input "b"). To do this I have used 4 NOT gates and store it in b_complement (array of wires) so I get the 1's complement of b. to get 2's complement I should add 1 to 1's complement of b, I have add it in the carry_in in the first full adder as shown in the picture above. Then we need to add the first input "a" and the 2's complement of b. I have do it using ripple adder by 4 full adders.

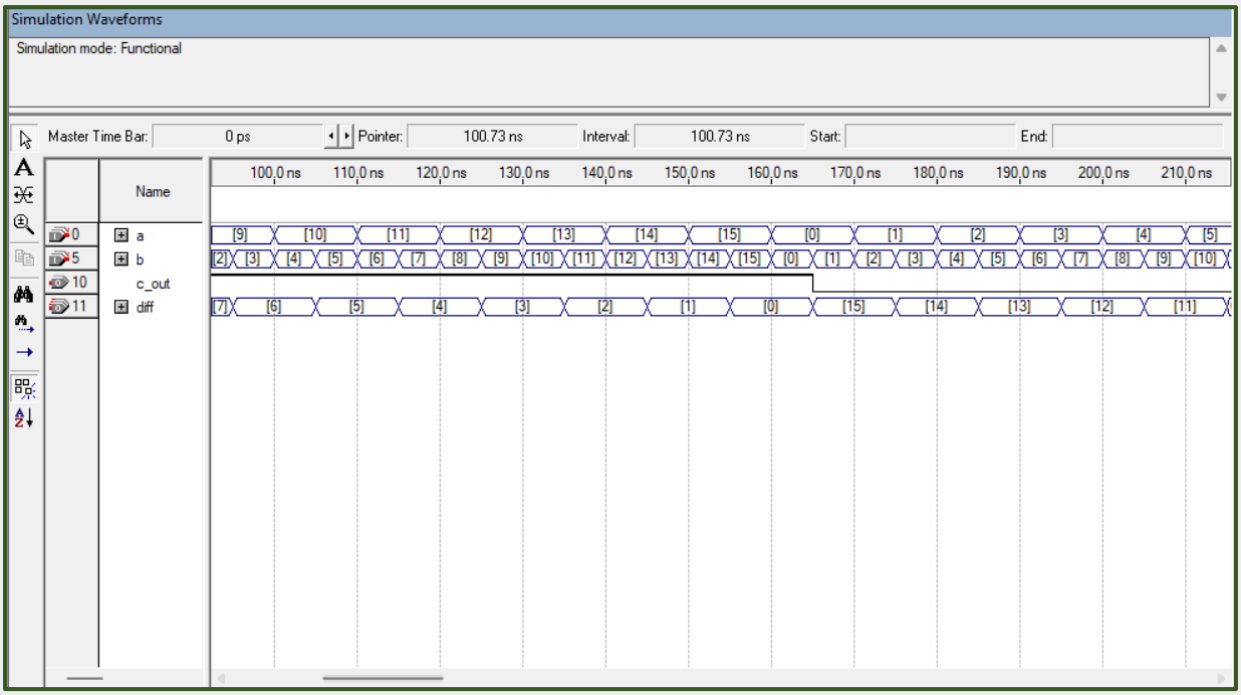

B4
A4
B3
A3
B2
A2
B1
A1
C4
Full Adder 4
C3
Full Adder 3
C2
Full Adder 2
C1
Full Adder 1
I
S4
S3
S2
S1

Symbol:


four_bits_subtractor
a[3..0]
b[3..0]
diff[3..0]
c_out
inst

Simulation result:





Test case: let a = 14 = $(1110)_2$ & b = 12 = $(1100)_2$ → 1's complement of b = $(0011)_2$

14 - 12: $a_0 = 0 + b'_0 = 1$ + carry in form first adder = 1 , $Sum_0 = 0$,$C_1 = 1$

$a_1 = 1 + b_1 = 1 + C_1 = 1$, $Sum_1 = 1$,$C_2 = 1$

$a_2 = 1 + b_2 = 0 + C_2 = 1$, $Sum_2 = 0$,$C_3 = 1$

$a_3 = 1 + b_3 = 0 + C_3 = 1$, $Sum_3 = 0$,$C_4 = 1$ so sum = $(0010)_2$ = 2 and carry out = 1 which the same in the second picture above.

ALU using DataFlow:

code implementation:

```
module ALU_dataFlow(a,b,selection,c_out,result);
input [3:0] a,b;                        // two inputs to make operations on them
input [2:0] selection;                  // the operation code to select the operation to do
output c_out;                          // carry out
output [3:0] result;
wire [3:0] Subtraction,Addition,Bitwise_OR,Bitwise_AND; // wires to store the result from each operation
wire add_c_out,sub_c_out;   // wires to store carries from addition & subtraction
// calling pervious operations
four_bits_subtractor(a,b,Subtraction,sub_c_out);
four_bits_adder(a,b,1'b0,Addition,add_c_out);
ors(a,b,Bitwise_OR);
ands(a,b,Bitwise_AND);

assign result = (selection == 3'b 000)? Subtraction:
                (selection == 3'b 001)? Addition:
                (selection == 3'b 010)? Bitwise_OR:
                (selection == 3'b 011)? Bitwise_AND:4'b0000; // set result equal zero if non of above operations is selected

 assign c_out = (selection == 3'b 000) ? sub_c_out :  // Carry out from Subtraction
                   (selection == 3'b 001) ? add_c_out :  // Carry out from Addition
                   1'b0;                                 // c_out for other operations

endmodule
```

As shown in the picture above, I have used Data flow modeling to implement ALU and for connecting the modules.

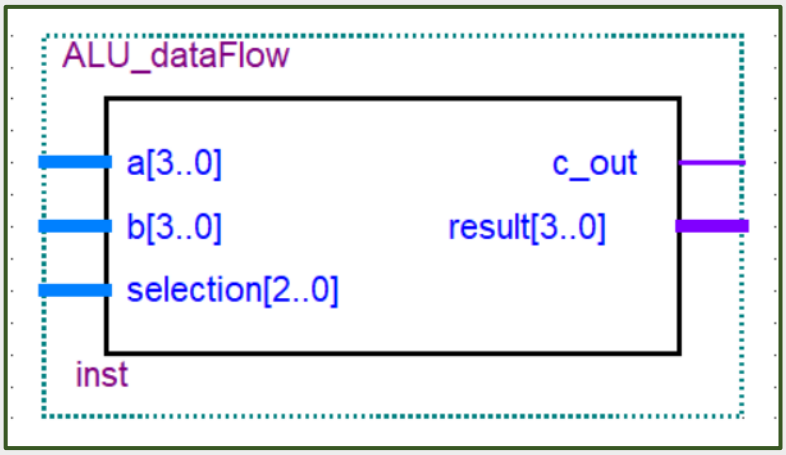
The ALU will choose the operation to do it on the inputs a & b based on the selection code entered (OpCode).

“Selection” is a 3-bit control input (OpCode) to select the operation: • 3'b000: Subtraction • 3'b001: Addition • 3'b010: Bitwise OR • 3'b011: Bitwise AND. Then Output the result (Result) of the selected operation.

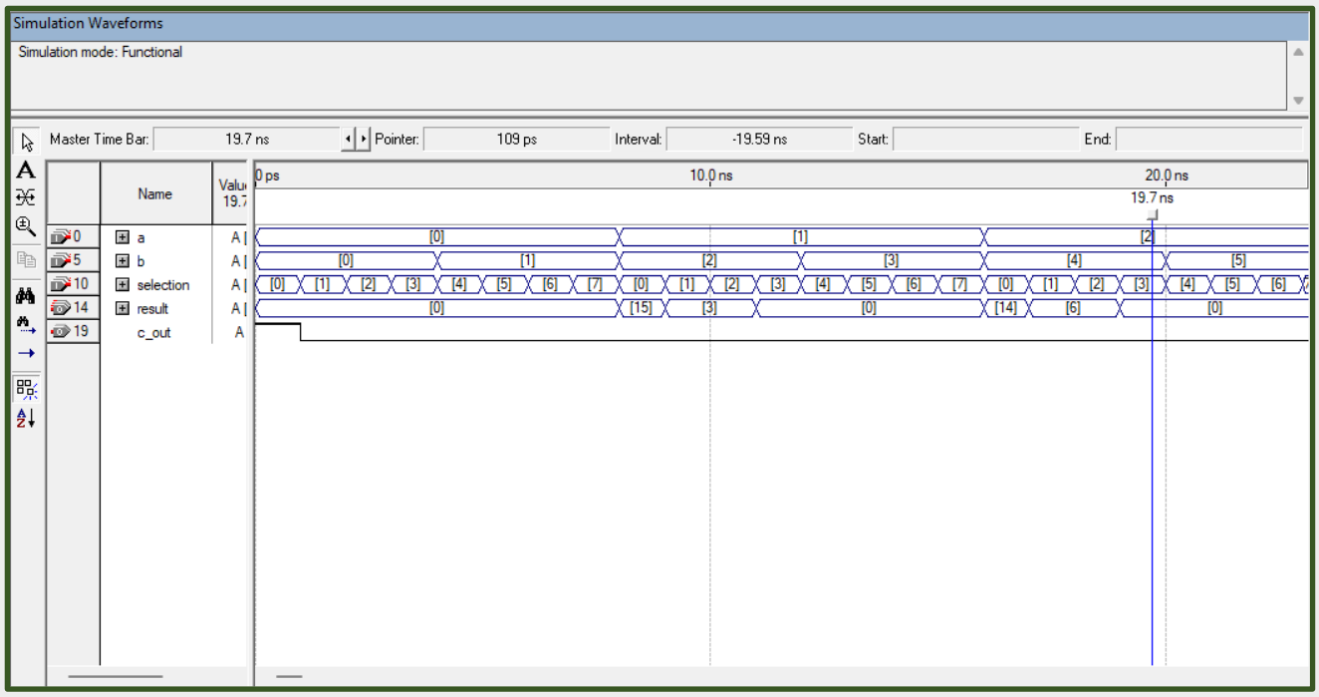
I have used wires (Addition,subtraction,Bitwise_OR,Bitwise_AND) to store the output from each operation to set the result later based on the operation selected by the OpCode (named “selection” in my code). also wires(add_c_out,sub_c_out) to store the carry out from adding & subtraction operations to set the final

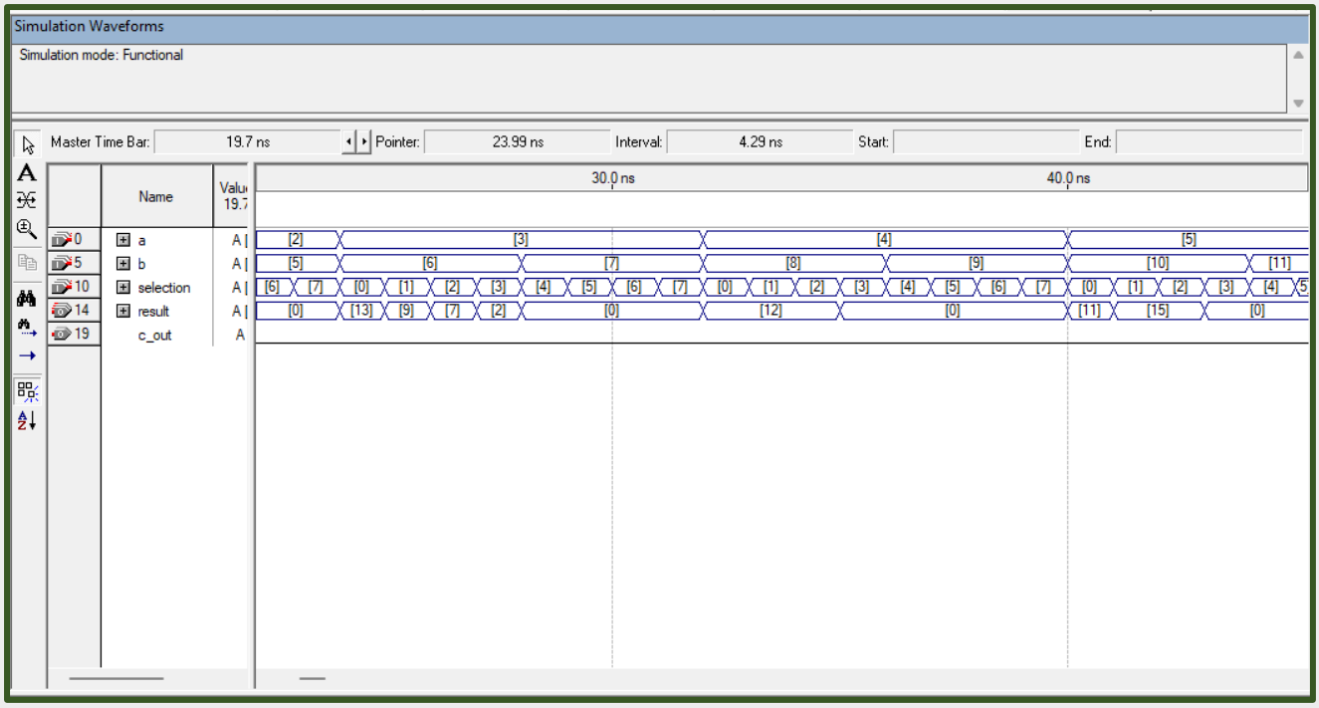
carry out equal one of these operations(adding & subtraction) or set it equal to zero if neither operation had selected.

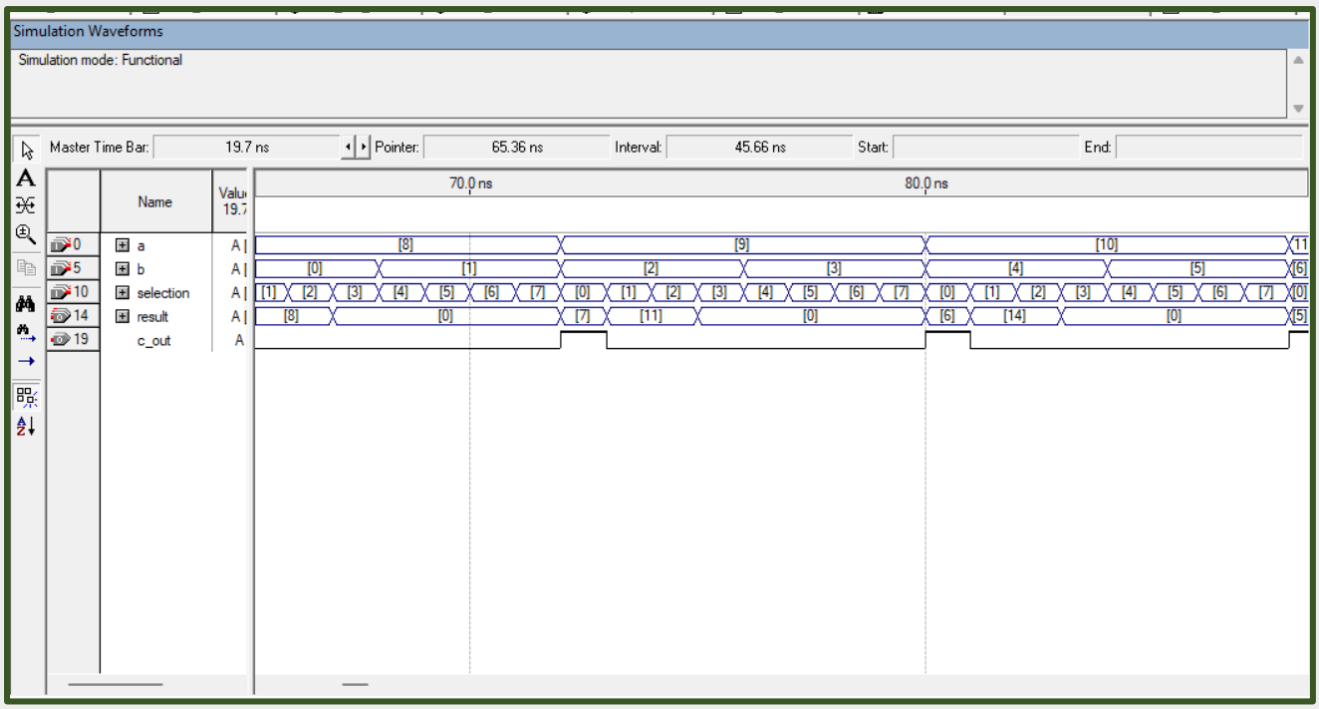
Symbol:



Simulation result:







Test case:

Let a = 9 = $(1001)_2$ & b = 2 = $(0010)_2$

When selection = 000 operation is a – b:

1001 1001 9 -2 = 7

0010 – → 1110 +

10110 = 7 & carry = 1

When selection = 001 operation is a + b:

1001 9+2 =11

0010 +

1011 = 11

When selection = 010 operation is a OR b:

1001 9 OR 2 =11

0010 OR

1011

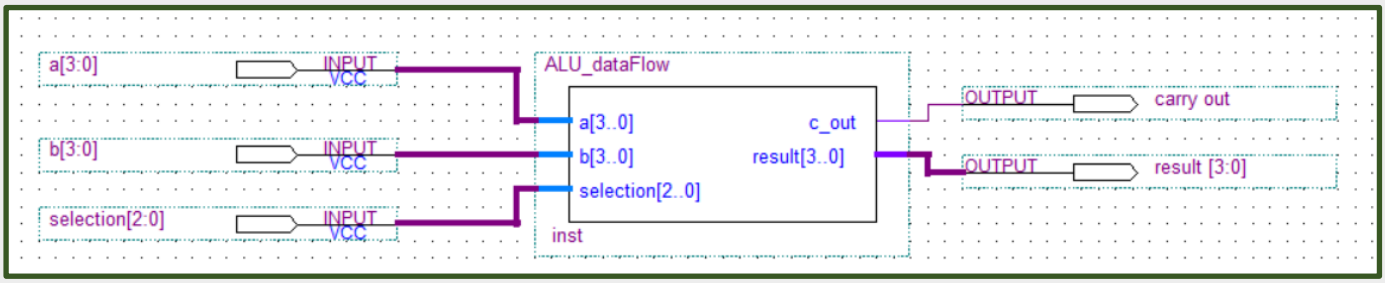
When selection = 011 operation is a AND b:

1001 9 AND 2 =0

0010 AND

0000

block diagram of ALU_dataFlow:



ALU using Behavioral:

Code implementation:

```
module ALU_Behavioral(a,b,OpCode,result,c_out);
input [3:0] a,b;        // two inputs to make operations on them
input [2:0] OpCode;     // the operation code to select the operation to do
output reg c_out;                                // carry out from Addtion & carry out from subtraction
output reg [3:0] result;
wire [3:0] Subtraction,Addition,Bitwise_OR,Bitwise_AND;// wires to store the result from each operation
wire sub_c_out,add_c_out;  // wires to store carries from addition & subtraction
// calling pervious operations
four_bits_subtractor (a,b,Subtraction,sub_c_out);
four_bits_adder (a,b,1'b0,Addition,add_c_out);
ors (a,b,Bitwise_OR);
ands (a,b,Bitwise_AND);

always@(*) begin
     case(OpCode)           // cases to set the result based on OpCode entered
        3'b000:begin
        result = Subtraction;
        c_out = sub_c_out;          // set the carry out equal to carry out of Subtraction when choose it
        end
        3'b001:begin
        result = Addition;
        c_out = add_c_out;          // set the carry out equal to carry out of Addition when choose it
        end
        3'b010:begin
        result = Bitwise_OR;
        c_out = 1'b0;               // set the carry out equal 0
        end
        3'b011:begin
        result = Bitwise_AND;
        c_out = 1'b0;               // set the carry out equal 0
        end
    default:
         result = 4'b0;
    endcase
    end
endmodule
```

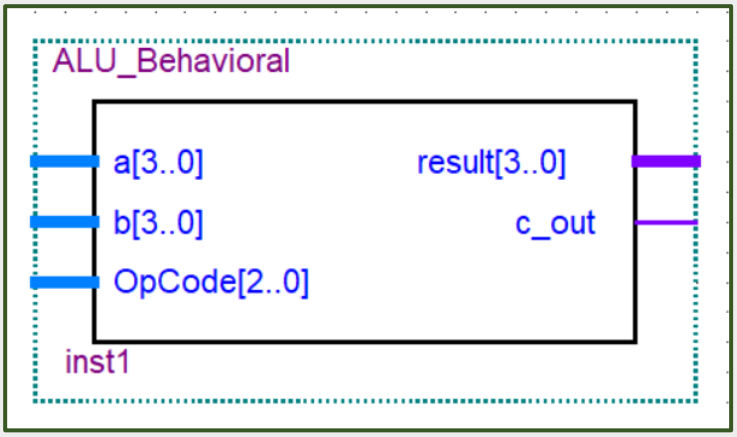
As shown in the picture above, I have used Behavioral modeling to implement ALU and for connecting the modules.

The ALU will choose the operation to do it on the inputs a & b based on the selection code entered (OpCode).

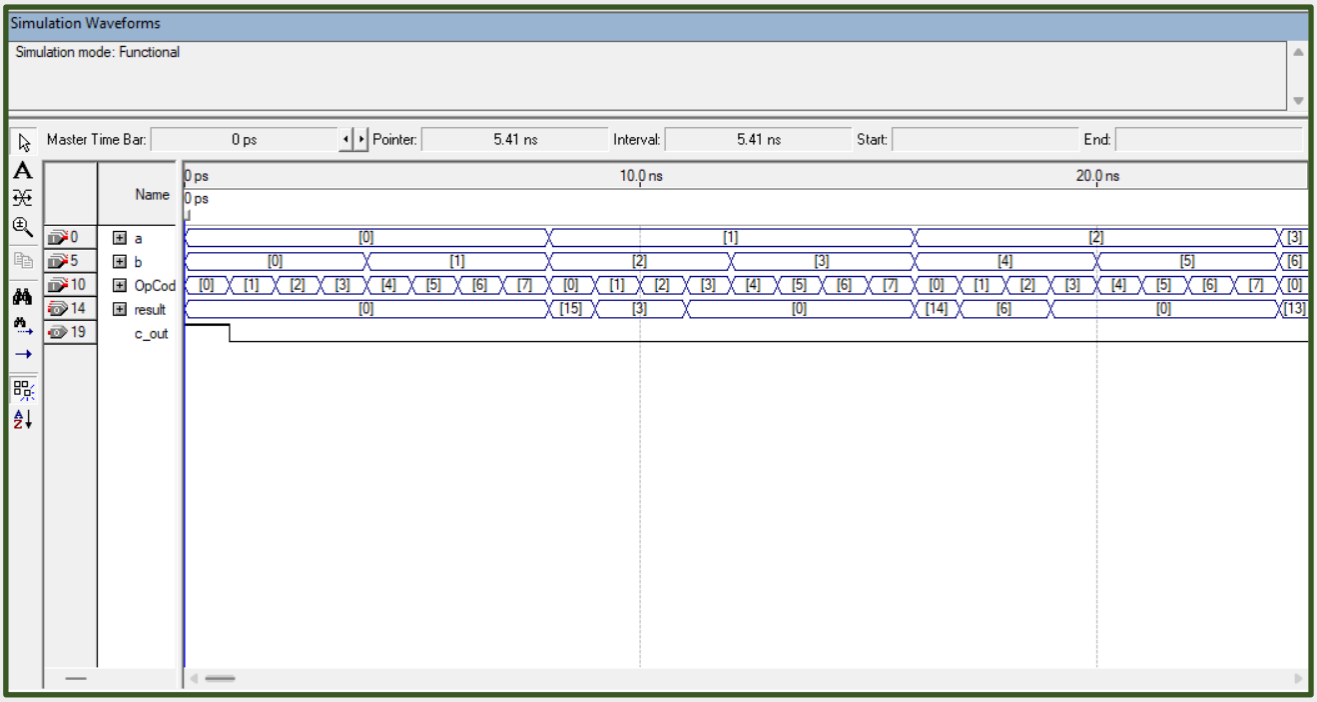
“Opcode” is a 3-bit control input (OpCode) to select the operation: • 3'b000: Subtraction • 3'b001: Addition • 3'b010: Bitwise OR • 3'b011: Bitwise AND. Then Output the result (Result) of the selected operation.

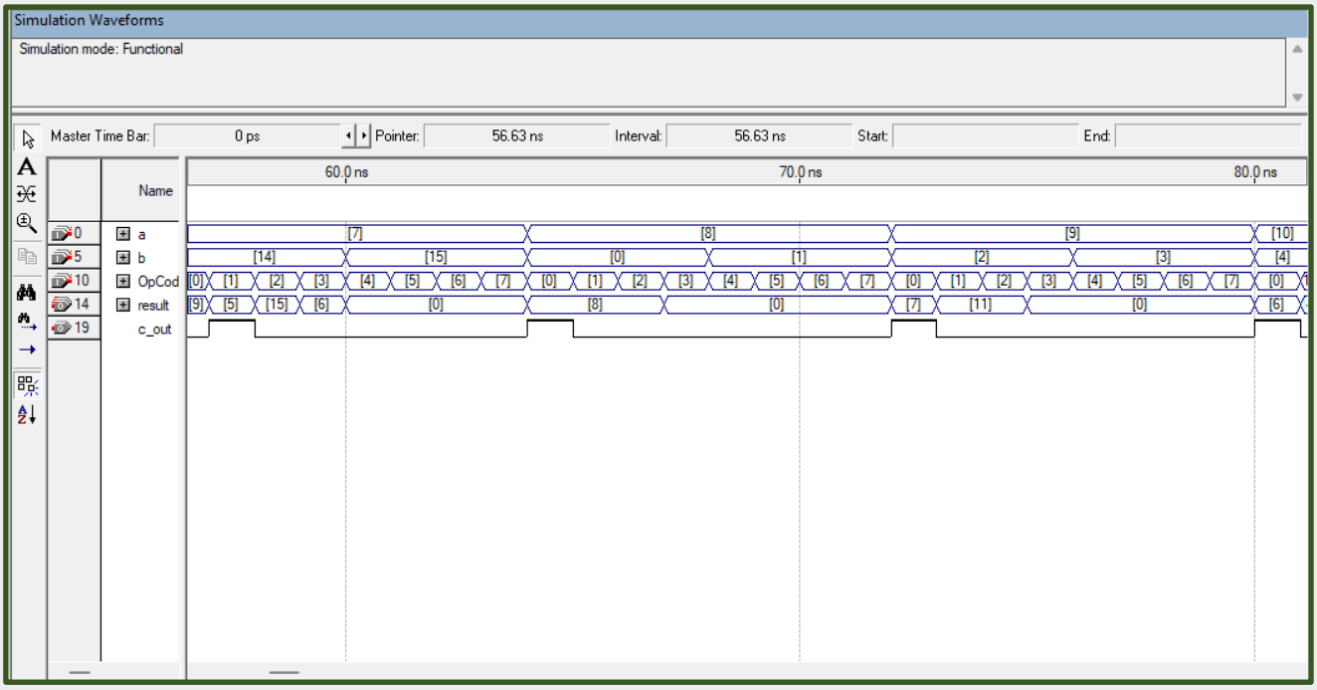
I have used wires to store the output from each operation to set the result later based on the operation selected by the OpCode. Moreover, I have used wires(add_c_out,sub_c_out) to store the carry out from adding & subtraction operations to set the final carry out equal one of these operations(adding & subtraction) or set it equal to zero if neither operation had selected.

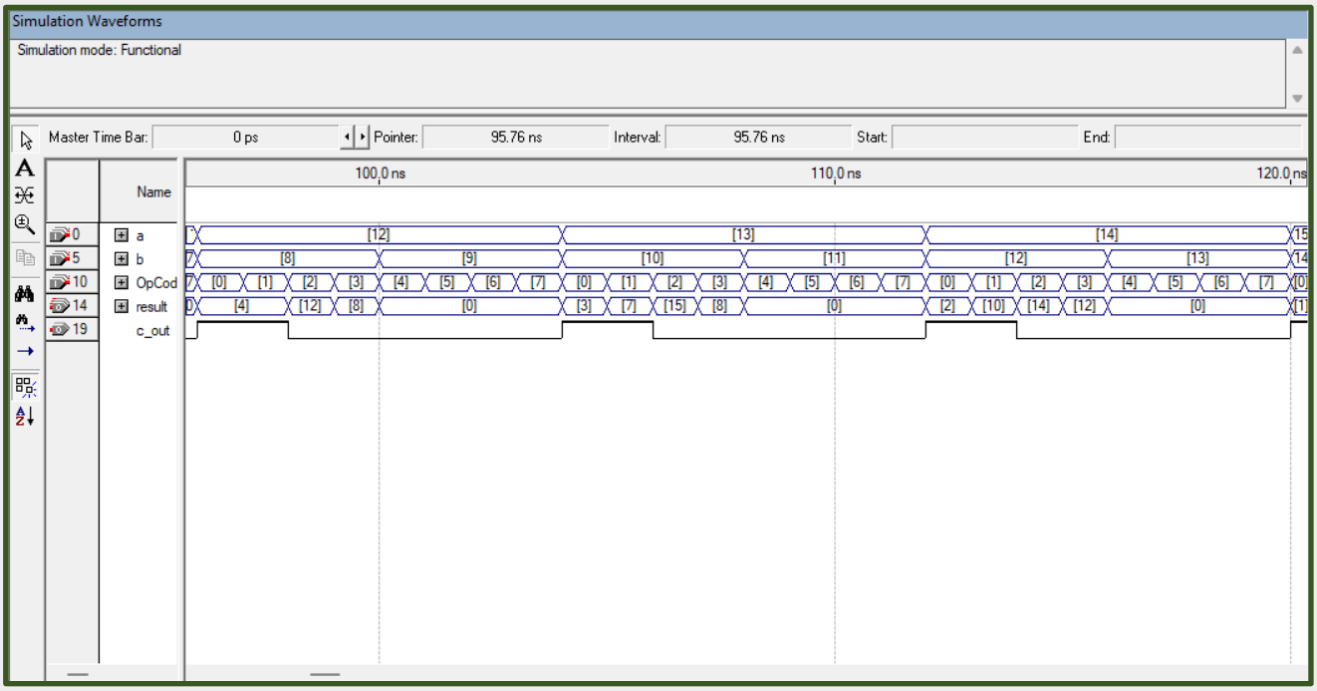
Symbol:



Simulation result:







test case:

let a = 13 = (1101)2 , b = 10 = (1010)2

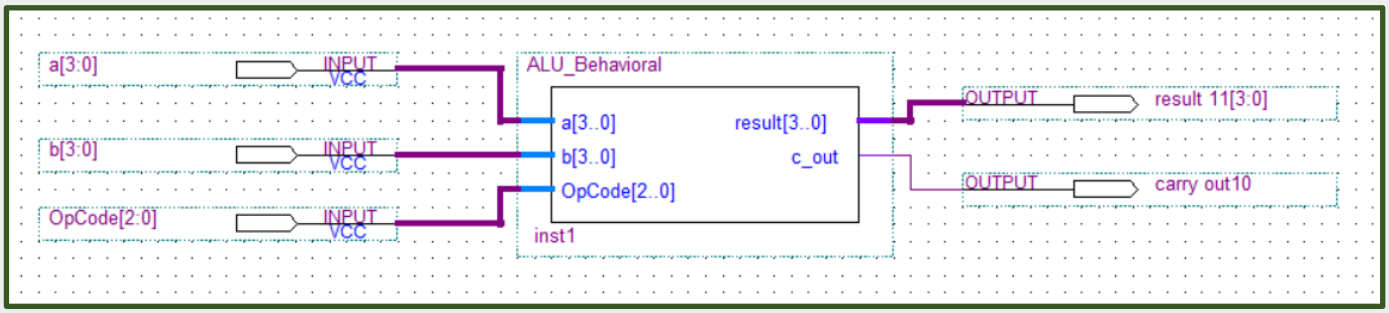
When selection = 000 operation is a – b:
1101 1101 13 -10 = 3
1010 – → 0110 +
10011 = 3 & carry = 1

When selection = 001 operation is a + b:
1101 13 + 10 = 23
1010 +
10111 = (23) = 7 & carry 1

When selection = 010 operation is a OR b:
1101 13 OR 10 =15
1010 OR
1111 = 15

When selection = 011 operation is a AND b:
1101 13 AND 10 =8
1010 AND
1000 = 8

Block diagram of ALU_Behavioral:



Conclusions:

In conclusion, This project taught us the importance of the ALU(Arithmetic Logic Unit) and its working principle, as it is capable of performing basic arithmetic and logical operations by receiving the relevant inputs and a code (OpCode) to select the required operation. We created a model for the ALU (Like a simulator) with simple operations, which are addition, subtraction, bitwise AND, and bitwise OR, using the Verilog programming language and in different types: structural modeling(Gate-level), behavioral modeling , and dataflow modeling with the Hierarchical Design method.